

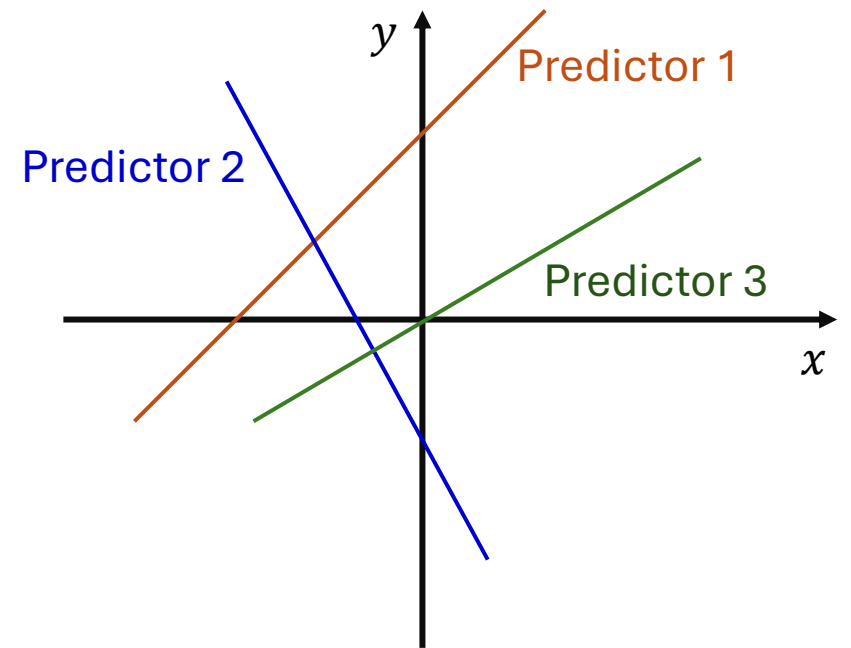
CS 767

Class 2

Linear Regression, and Cross-Validation

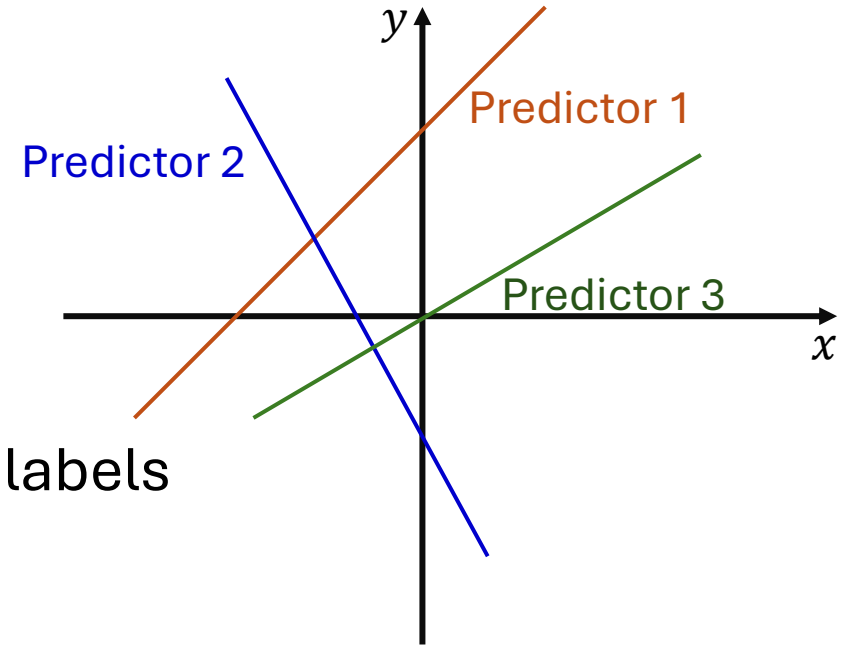
Recap: Linear Regression

- **Basic idea:** model the relation between features and labels as a LINEAR function
- We're given a dataset $\mathcal{D} = (x_i, y_i)_{i=1}^N$
 - features x_i (1-dimensional)
 - actual labels y_i
 - Model: $\hat{y}_i = w_0 + w_1 x_i$
- Collection of regressors: all straight lines.
 - Let's denote this collection $\mathcal{H}$
 - **Every line in $\mathcal{H}$ is completely specified by w_0, w_1**



Recap: Linear Regression

- Which model (line) in $\mathcal{H}$ is the best?
- “Best” with respect to what?
 - Could measure how close predictions are to actual labels
- $\mathcal{D} = \mathcal{D}_{train} \cup \mathcal{D}_{test}$
- SE: $\mathcal{L}(y, \hat{y}) = (y - \hat{y})^2$



$$w_0^*, w_1^* = \operatorname{argmin}_{w_0, w_1 \in \mathcal{H}} \frac{1}{N_{train}} \sum_{i \in \mathcal{D}_{train}} (y_i - \hat{y}_i)^2$$

Linear Regression (K features)

- **Basic idea:** model the relation between features and labels as a LINEAR function
- Given dataset $\mathcal{D} = (\mathbf{x}_i, y_i)_{i=1}^N$
 - features $\mathbf{x}_i = [x_1, x_2, \dots, x_K]$
 - labels y_i
 - Model: $\hat{y}_i = w_0 + w_1 x_{i,1} + \dots + w_K x_{i,K} = w_0 + \sum_{j=1}^K w_j x_{i,j}$
- $\mathcal{H}$ is now a collection of ***planes***.

Linear Regression (K features)

- **Simple Example:** house price prediction
 - California housing dataset
- $\mathcal{D} = (\mathbf{x}_i, y_i)_{i=1}^N$
 - features $\mathbf{x}_i = [\text{lat}, \text{lon}, \text{median age}, \text{total rooms}, \text{total bedrooms}, \text{ocean proximity}, \dots]$
 - labels y_i : median house value
 - $K = 9$ features
 - Model: $y_i = w_0 + \sum_{i=1}^9 w_i x_i$
- Squared loss = (actual price – predicted price)²

Polynomial Regression

- **Basic idea:** model the relation between features and labels as a POLYNOMIAL
 - Interaction terms
 - Nonlinearity
- Given dataset $\mathcal{D} = (x_i, y_i)_{i=1}^N$
 - Model: $\hat{y}_i = w_0 + w_1x_i + w_2x_i^2 \dots + w_dx_i^d$
- $\mathcal{H}$ is now a collection of ***polynomials***.

Polynomial Regression

- Given dataset $\mathcal{D} = (x_i, y_i)_{i=1}^N$
 - Model: $\hat{y}_i = w_0 + w_1 x_i + w_2 x_i^2 \dots + w_d x_i^d$
- $\mathcal{H}$ is now a collection of ***polynomials***.

- Training remains essentially the same!

$$w_0^*, \dots, w_d^* = \operatorname{argmin}_{w_0, w_1 \in \mathcal{H}} \frac{1}{N_{\text{train}}} \sum_{i \in \mathcal{D}_{\text{train}}} (y_i - \hat{y}_i)^2$$

Polynomial Regression

- With K features,

$$x_i = [x_{i,1}, \dots, x_{i,K}]$$

- Polynomial of degree d

- Interaction terms

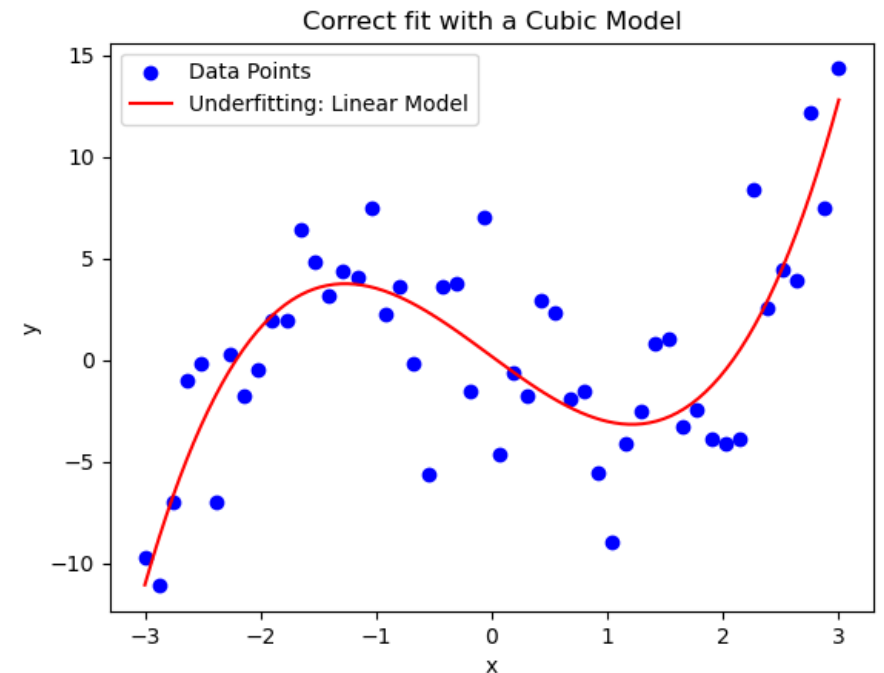
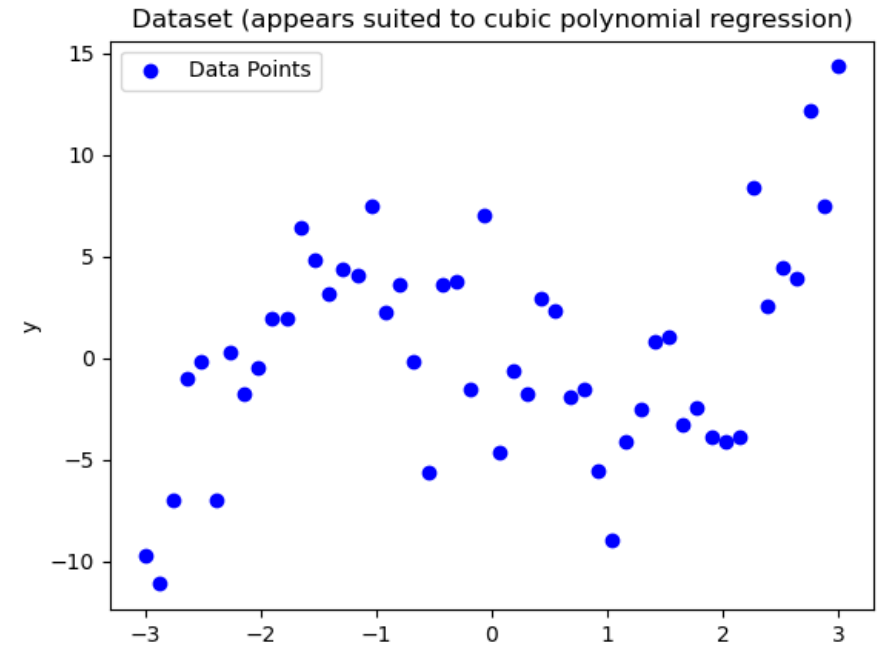
- $x_{i,1}^3 x_{i,4}^2$

- Total number of terms (including constant)

- $\frac{(K+d)!}{K!d!}$

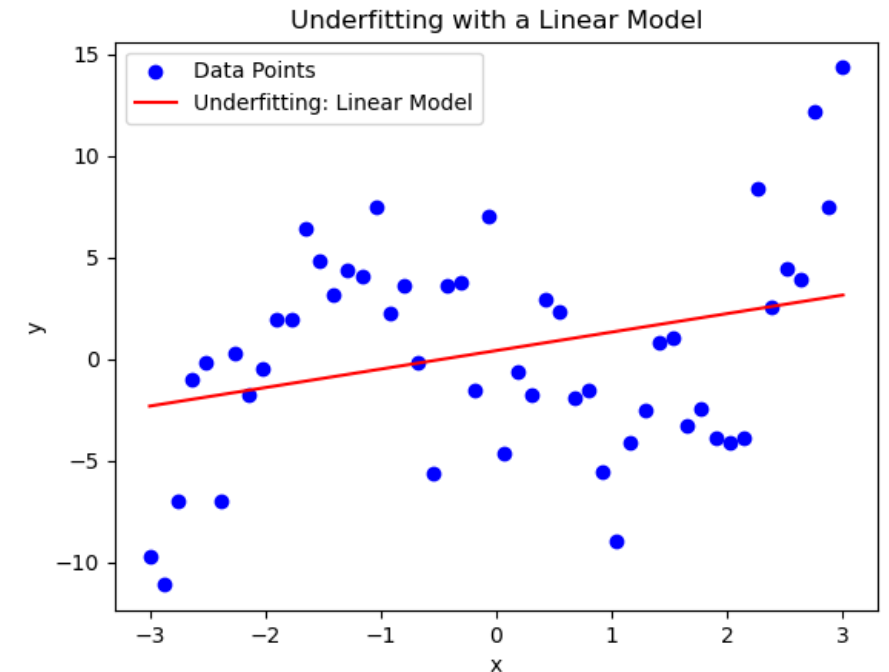
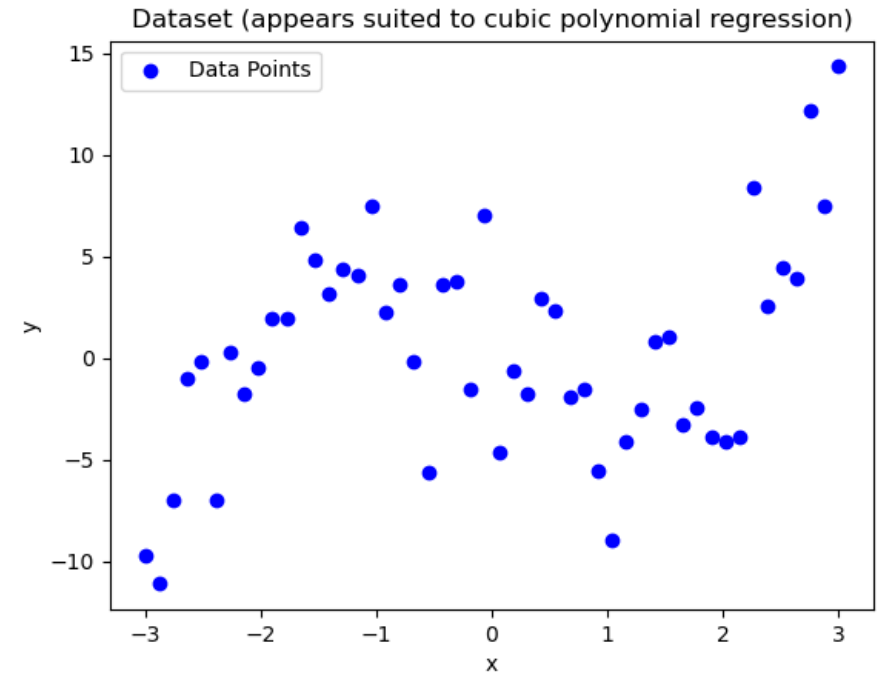
Overfitting vs Underfitting

- How does one choose the degree of said polynomial?
 - What prevents us from **increasing the degree without bound**?
- In general, increasing model complexity in search for better performance leads to loss of **generalizability**.



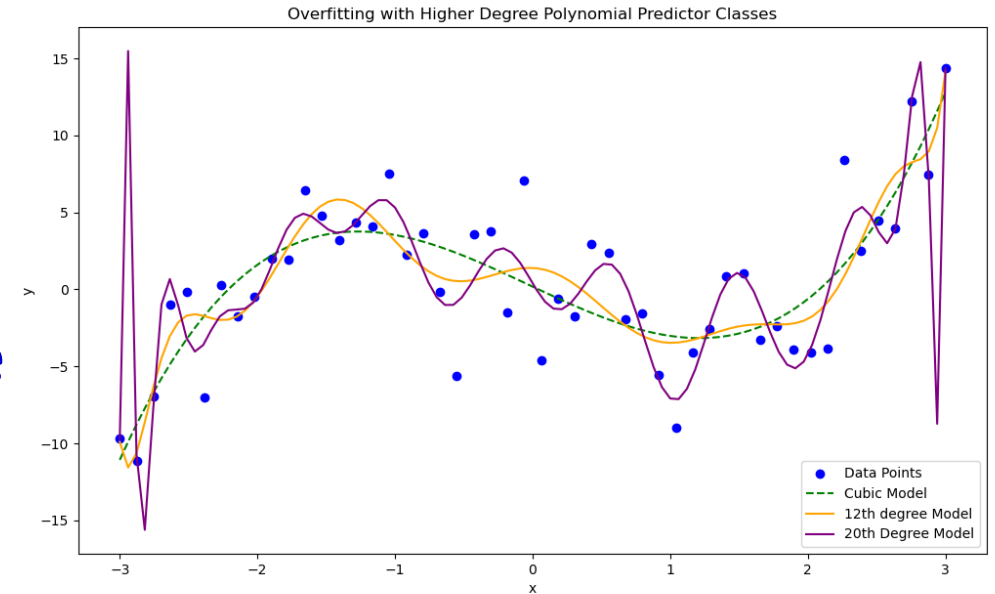
Overfitting vs Underfitting

- When a model **underfits**, it is too **simple** (e.g., using a linear model on highly nonlinear data)
 - fails to capture important relationships in the data, resulting in high bias.
- Systematic errors on **both training and test data**,
 - meaning the model misses the mark consistently.

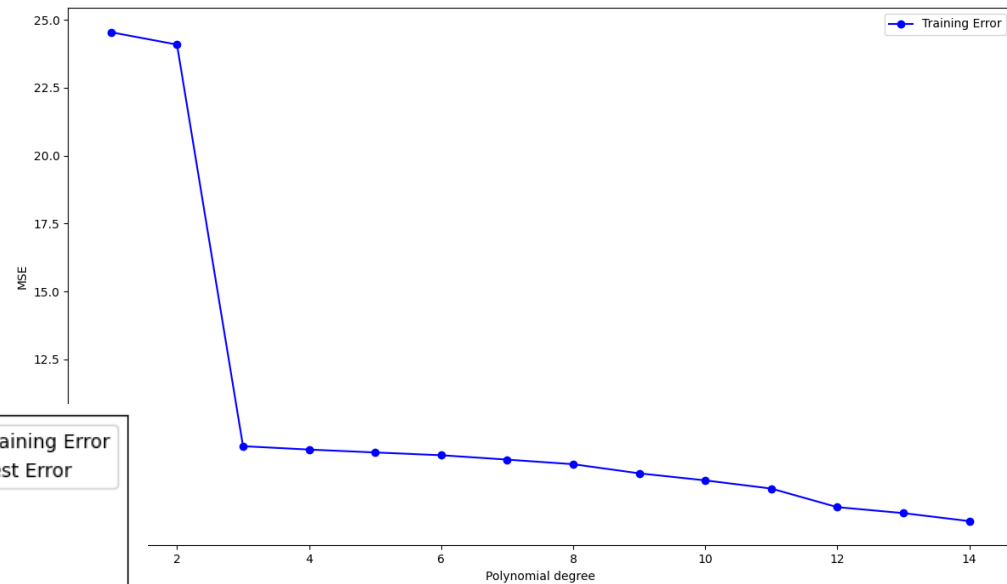
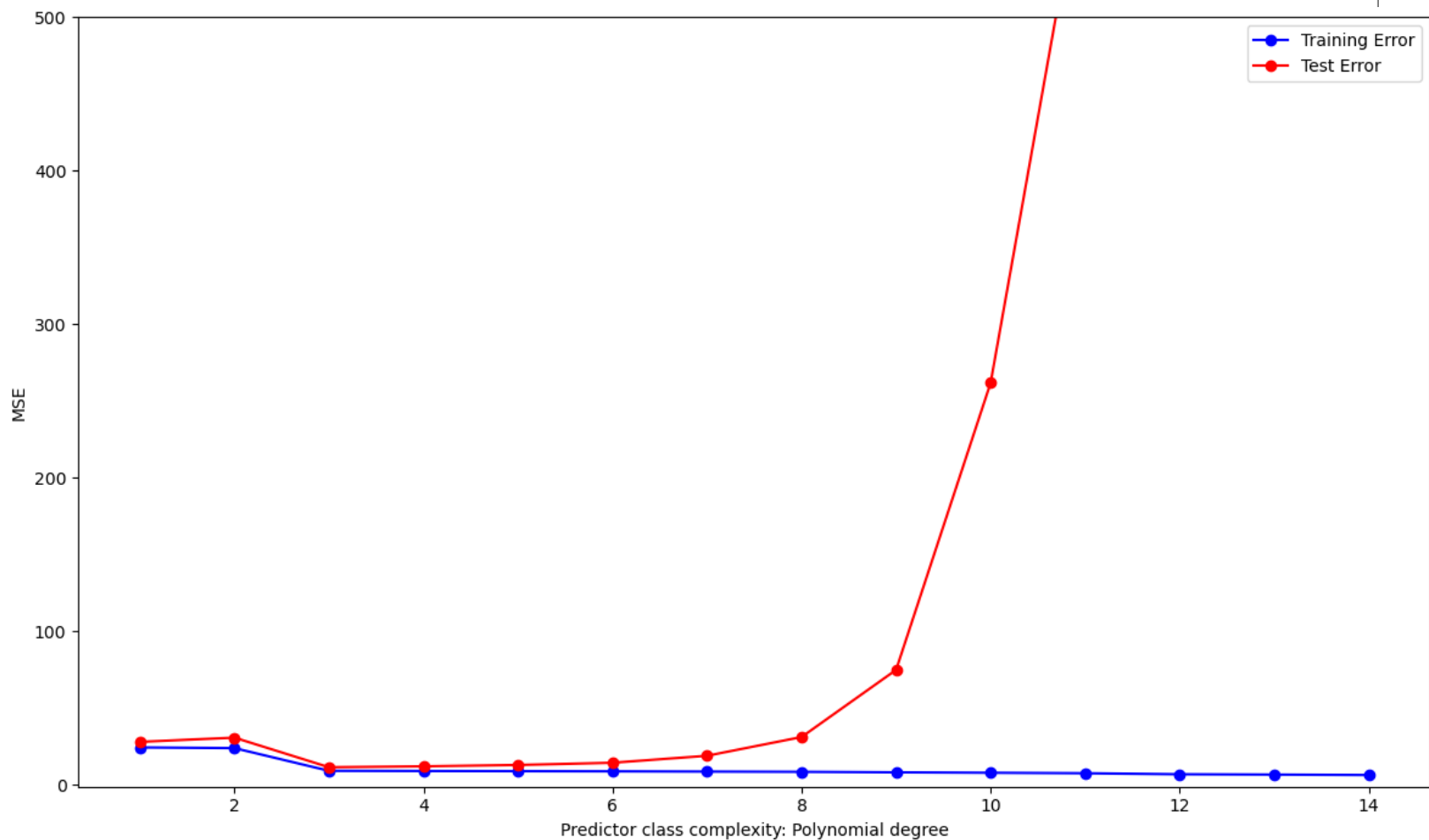


Overfitting vs Underfitting

- When a model **overfits**, tailors itself too closely to the training data, capturing **noise or random fluctuations** rather than the underlying pattern.
- Memorization
- **Low error on the training set but high error on the test set**
 - Model becomes **unreliable on new data** due to over-customization to the specific details of the training data.
 - Memorization



Choosing polynomial degree



Choosing polynomial degree

Outer loop: fix polynomial degree $d \in \{1, \dots, 14\}$

Inner loop: train model with degree d

compute train and test errors

compute average train and test errors

append to lists.

Regularization

- refers to a set of techniques used to **reduce overfitting** by adding a **penalty term** to the loss function.
- **Idea**: admit a small increase in train error to gain a large reduction in test error.

$$\mathcal{L}(\mathbf{w}) = \frac{1}{\mathcal{N}_{train}} \sum_{i=1}^{\mathcal{N}_{train}} (y_i - \hat{y}_i)^2 + \lambda \textit{Penalty Term}$$

- Regularized loss = Standard Loss + (Reg. coeff.) × Penalty term

Regularization

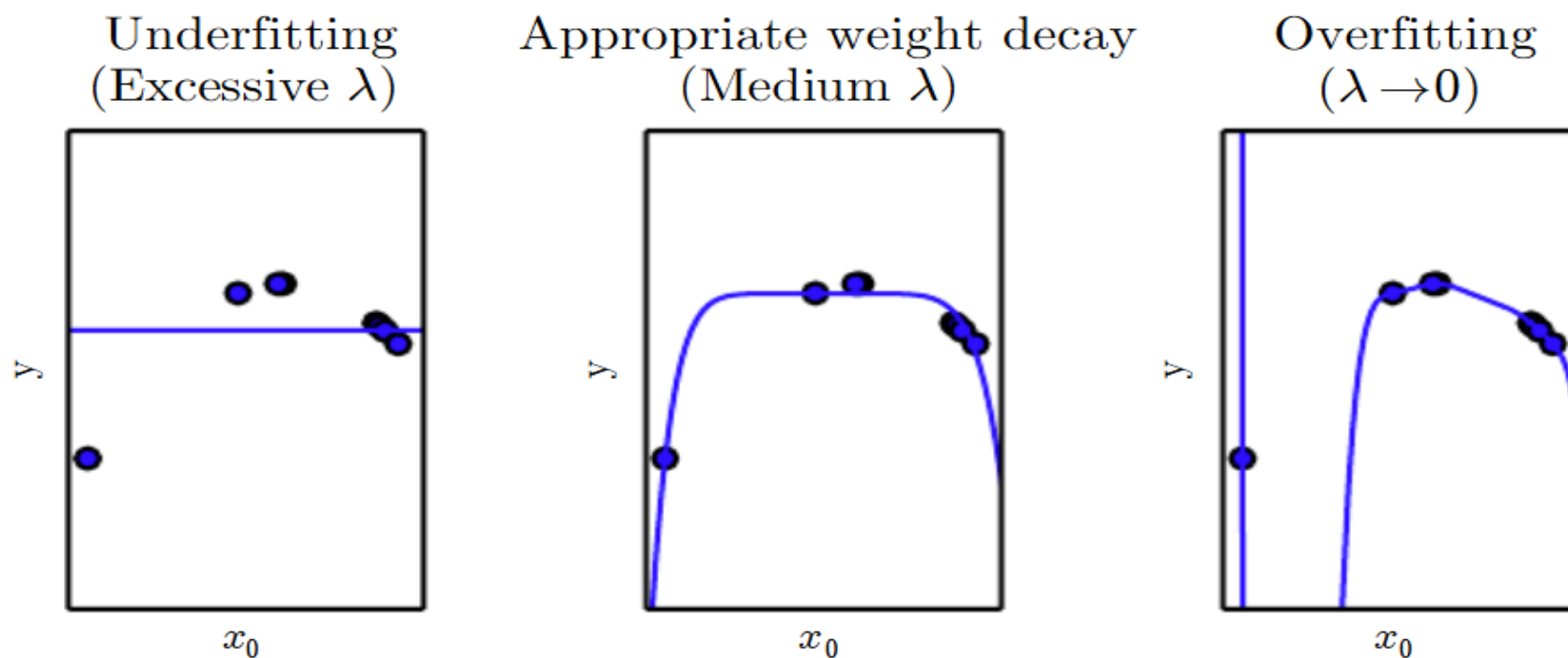


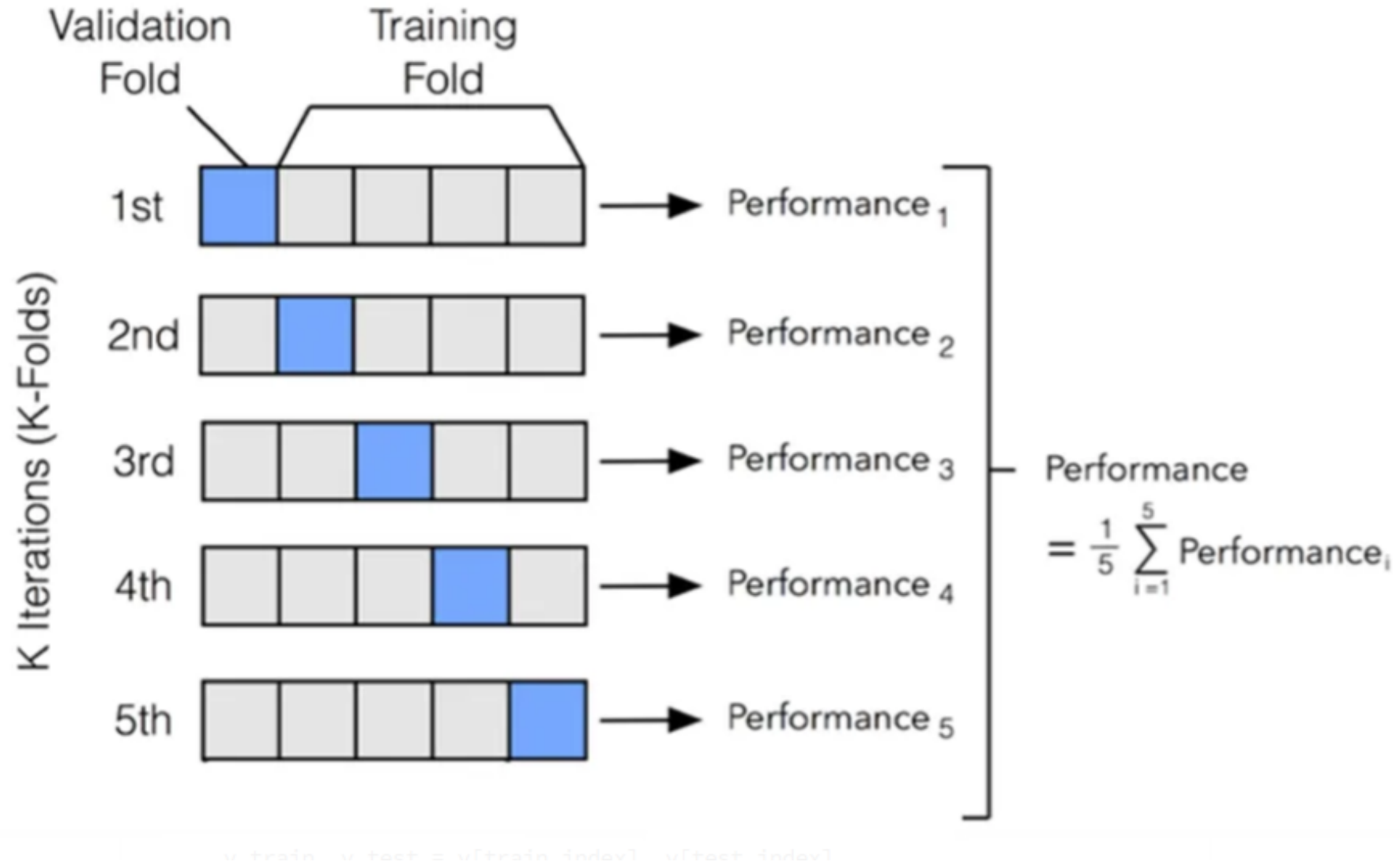
Figure 5.5: We fit a high-degree polynomial regression model to our example training set from figure 5.2. The true function is quadratic, but here we use only models with degree 9. We vary the amount of weight decay to prevent these high-degree models from overfitting.

(L-6) With increasing λ , we move from the model to the true function with increasing

Cross Validation

- **Motivation 1:** $\mathcal{L}(\mathbf{w})$ versus $\mathcal{L}(\mathbf{w}, \lambda)$
 - How will you find the optimal value of λ ?
- **Motivation 2:** to build a good model, we should use as much of the available data as possible for training.
 - Problem: data leakage.
- Split data into
 - Train
 - Validation
 - Test

Cross Validation



Logistic Regression: algorithm flow

